The default algorithm for shuffling shards is implemented by `LeastShardAllocationStrategy`, which shuffles shards from the regions with the most shards to the regions with the fewest shards. The default setting for rebalance, which you can find in reference.conf, is

```
akka.cluster.sharding.least-shard-allocation-strategy.rebalance-relative-
➥limit = 0.1
```

which means only 10% of the total shards may be moved per round. In general, this is good enough.

9.1.5 Passivation

Actors are very cheap to create—each is about 300 KB, so you can create thousands in 1 GB. But if you use them to store large objects, the memory requirements are quite different, and memory can be exhausted much faster. In these cases, it makes sense to *passivate* the entities: that is, to store them by specifying how long they should remain in memory after their last received message. The default is the following:

```
akka.cluster.sharding.passivate-idle-entity-after = 120s
```

However, passivation is disabled if `remember-entities` is enabled, which is not the case by default:

```
akka.cluster.sharding.remember-entities = off
```

When a passivated entity starts receiving new messages, the entity's mailbox is not yet active. Instead, the system buffers the messages and stores them until the entity is active again. But the buffer has a limit. Remember that only messages sent through an SR count, not direct messages to the underlying `ActorRef`.

9.1.6 Remembering entities

After a rebalance or entity crash, entities are automatically reloaded into memory if they have a backup. You need to set `akka.cluster.sharding.remember-entities -store` to one of these to create an entity backup: `ddata` or `eventsourced`. Both settings also disable passivation. Akka Distributed Data (`ddata`) is a structure that can handle concurrent updates without coordination and is the default setting, while `eventsourced` keeps the events in a persistent storage.

NOTE Messages are sent with at-most-once semantics by default, and the order is maintained by the SR. All messages sent to the same SR maintain the order of receipt.